

A Neural Microkernel: Interruptible, Verified Inference with a Self-Extending Deterministic Plane and a Scheduler-Owned Branching Policy

Pierre Lamy

with Claude Code (Anthropic, Opus 4.8)

2026-06-23

Abstract

Large language models are deployed as uninterruptible black boxes: once a generation begins it runs to completion, its intermediate state is neither inspectable nor revertible, and any claim it makes about the world is taken on faith. We report an engineering study that wraps a real Mixture-of-Experts model (Cohere2-MoE / “North”, run locally through MLX) in an operating-system-style *microkernel* whose organizing principle is a privilege boundary: the model *proposes*, a trusted deterministic plane *disposes*. The kernel contributes (i) a fail-closed scheduler with a three-gate admission contract and a capability/syscall model; (ii) a family of forward-pass mechanisms — a commit trap, KV-cache park/resume, expert-isolation, speculate/verify/rollback, deterministic interruption, and a *hidden-state result injector* — each proven to bit-exactness against real model activations; (iii) a *self-extending deterministic plane* in which the kernel detects a missing primitive, has the model synthesize it, and admits it only after a hold-out gate proves it correct on thousands of novel instances against an independent oracle (with a placebo control that rejects fail→pass-by-luck); (iv) an escalation solver that climbs greedy → temperature-diverse → repair → web-research → honest abstention; (v) a small, hard-bounded, always-consulted persistent memory governed by a self-describing policy object; (vi) a *scheduler-owned dynamic branching policy* (the model proposes route scores, the kernel disposes how many continuations k to run) that governs every branch point — escalation width, fact dispatch, concurrency, verification depth, and capability proposal — thin by default and widening only where calibrated uncertainty and host cost permit; and (vii) a *capability-multiplier loop* that delivers a hold-out-proven fact to the model as tokens and lifts its accuracy on a task it cannot do unaided by $\sim 9.5\times$, governed by the same policy. Two results are reported as honest negatives: a Mixture-of-Experts gate-entropy signal that, when calibrated, proves *inverted* and is therefore left unwired; and hidden-state *fact-use* (injecting a value the model must compute *with*), which fails through the residual channel — the same fact delivered as *tokens* succeeds at 1.0 — a clean delineation of what residual injection can and cannot do. Validation comprises 28 CPU test suites (all passing), eight tensor-level invariant batteries that hold to bit-exactness on the live model, a 420-task coding

benchmark run *through the kernel* (418/420), adversarial multi-agent bug sweeps, hold-out-proven solutions to previously-failed tasks, and live North demonstrations of the branching policy and the capability multiplier. We are explicit about what is and is not integrated: the bit-exact forward-pass mechanisms are proven in isolation, the branching/dispatch policy is integrated and live, and the raw-Metal GPU mega-kernel fusion is recorded as a deliberate *nice-to-have, not built*.

1. Introduction

A modern language model is, operationally, a coroutine you cannot pause. It is invoked, it emits tokens until a stop condition, and it returns. While it runs there is no supervisor: no way to checkpoint the partial computation, no way to roll back a step that violated an invariant, no way to interpose a deterministic check between “the model decided x ” and “ x takes effect.” Worse, the model’s outputs are accepted on trust. When a model writes a helper function, or claims a fact, or reports that a test passed, nothing in the standard inference stack distinguishes a correct result from a confident hallucination.

Operating systems solved the analogous problem for untrusted user code decades ago. User processes do not get to decide whether their `write()` succeeds; they *request* it across a privilege boundary, and a trusted kernel — holding the only handle to the hardware — validates and disposes. The kernel does not need to *trust* the process because it *checks* the process. This paper asks what it takes to put that boundary around, and eventually inside, neural inference. The thesis, refined over the course of the work, is a single sentence: **the learned component proposes; a thin, general, trusted kernel disposes**. Critically — and this is the insight that shaped the architecture — *validation is a syscall*. Whether a generated primitive is correct, or a candidate fact is grounded, cannot be adjudicated by the same weights that produced it; it requires an independent, deterministic authority outside the model.

This report documents a working system built around that thesis and the experiments that validate it. Our contributions are:

1. **A fail-closed scheduler and capability model** (§2.2) with a three-gate admission contract: an operation executes only if it is registered, its capability has been granted, and a validator is installed — defaulting to refusal on any omission.

2. **Forward-pass kernel mechanisms** (§2.4) — commit trap, KV park/resume, switch-MLP expert isolation, speculate/verify/rollback, deterministic interrupt, ring-buffer-crossing rollback, and a hidden-state result injector — each validated to *bit-exactness* (residual $\equiv 0$) against the activations of a real MoE.
3. **A scheduler-owned dynamic branching policy** (§2.5): the branching factor k at every continuation point — how many drafts, tools, verifiers, agents, or repairs to run — is the *kernel’s* decision, disposed from route scores the model proposes, gated by calibrated uncertainty and host cost, thin by default. This is “learned-proposes / kernel-disposes” applied to control flow itself.
4. **A self-extending deterministic plane** (§3) governed by a hold-out gate: the kernel grows new, *proven* capabilities at runtime without ever trusting model output, because the trust verdict is computed in the parent against an independent oracle, never reported by the child; and its capability proposal is itself k -governed (§3.6).
5. **A capability-multiplier loop** (§4): the kernel computes a hold-out-proven fact, delivers it to the model *as tokens*, and the model — which could not produce it unaided — now uses it; this is wired into a live dispatch path (a capability-gated scheduler op) whose *propose* half is a learned in-weights router.
6. **A persistent, hard-bounded, always-consulted memory** (§5) carrying a self-describing governance object.

We are deliberately conservative about claims, and we report two negative results as carefully as the positive ones (§2.5, §4.2). §6 separates what is proven at the tensor level from what is integrated into the running loop, and §7 states the limitations plainly — including the explicit decision *not* to build the GPU mega-kernel.

2. Architecture

2.1 Three planes

The system is organized into three planes that mirror the proposer/disposer split:

- **The neural plane** — the model’s forward pass. It is the *proposer*. Its outputs (next-token logits, expert routings, generated text) are treated

as untrusted proposals.

- **The scheduler plane** — a deterministic task graph that admits, sequences, parks, resumes, and commits units of work. It is the *dispatcher*: it decides what runs, *how many* run (§2.5), and enforces the contracts under which they run.
- **The deterministic plane** — pure, verifiable executors (parsers, transforms, validators, oracles, and the hold-out gate). It is the *disposer*: it is the only plane permitted to pronounce a result correct.

The privilege boundary runs between the neural plane and the other two. Anything the model produces must cross a gate to take effect.

2.2 The scheduler ABI and the three-gate contract

The scheduler exposes a deliberately small interface. Executors are registered as `register_executor(kind, fn)` where `fn(sched, task, inputs) -> (result, ok, ValidationLevel)`; validators as `register_validator(key, fn)` where `fn(result, task) -> bool`. Admission (`admit`) is **fail-closed** behind three independent gates:

1. **Operation exists** — the `kind` has a registered executor.
2. **Capability granted** — the task's declared capabilities are all present in the scheduler's grant set.
3. **Validator installed** — a validator is registered for the task's contract key, at a level no higher than the ceiling permitted for that operation.

A task that omits any of the three is rejected, not run. This is the structural reason the kernel can host side-effecting tools without being compromised by them: a tool with no granted capability, or no validator, simply cannot be admitted. Side-effecting tools additionally register a *syscall-op* descriptor (`_syscall_ops[name] = {capability, max_level}`), and admission enforces that the declared validator level does not exceed the tool's ceiling — so an I/O tool can never be silently promoted to a higher trust level than its capability warrants. A subtle ABI point, relied on by §4.3: the node's verification level is set by its *registered validator*, not by the `ok`/level an executor returns; the executor's tuple is the shape, the validator is the authority.

2.3 The trust spine: two tiers

The kernel recognizes exactly two tiers of trust, and never confuses them:

- **PROVEN (pure deterministic primitives).** A primitive earns this tier only by passing the hold-out gate (§3.1): correctness demonstrated on thousands of novel instances against an independent oracle. These are pure functions; they have no I/O and cannot lie about their effects.
- **CHECKED (syscall-tier tools).** Anything that touches the world — disk, network, the system clock, a subprocess — is capability-gated and its *result* is validated, but it is never marked PROVEN. The kernel cannot prove a network fetch “correct”; it can only check that the call was authorized and the result well-formed.

This distinction is enforced mechanically by the admission gates, not merely documented.

2.4 Forward-pass mechanisms

Six mechanisms give the kernel, in principle, OS-like control over a generation at the granularity of a single forward pass. Each is validated by an *invariant battery* — a test that asserts a residual is exactly zero on real model activations, not merely small:

- **Commit trap** — a checkpoint/abort primitive: a forward step can be speculatively taken and then either committed or reverted, with the post-revert state bit-identical to the pre-step state.
- **KV park/resume** — the key/value cache of a paused generation can be serialized (“parked”) and later resumed such that continuation is bit-identical to never having paused.
- **Switch-MLP expert isolation** — in the MoE, masking the expert router so that only a chosen subset of experts fires produces activations identical to an independent reference computation over that subset (the masker ports the native `cohere2_moe` routing verbatim, rather than re-deriving it).
- **Speculate / verify / rollback** — a draft continuation can be proposed, verified against a condition, and rolled back on failure, leaving no residue in cache or RNG state.
- **Deterministic interrupt + ring-cross rollback** — a generation can be interrupted at an arbitrary step and rolled back deterministically, *including* the case where the interrupted span crosses a KV ring-buffer boundary (the >4096-token restore path), which is the hard case for cache restoration.

- **Hidden-state result injector (Mode B)** — a zero-init residual sidecar (`ResidualGateway`) that, disabled, is bit-identical to the base model, and when armed writes a result-conditioned vector into the residual stream. Its disabled-identity invariant holds to $\text{residual} \equiv 0$ on the live model; what it can and cannot do when *armed* is the subject of §4.2 (a token-free value-delivery channel that works, and a fact-reasoning channel that does not).

Eight batteries (`inv_trap_plane`, `inv_kv_park`, `inv_switch_isolation`, `inv_speculate_rollback`, `inv_det_interrupt`, `inv_rollback_ringcross`, `inv_mode_b`, `inv_moe_entropy`) report $\text{residual} \equiv 0$ on the live North model.

Scope, stated honestly. These mechanisms are proven *in isolation*, at the tensor level. The integrated runtime today schedules at whole-generation granularity — it wraps complete generations rather than trapping every forward pass — and the in-model commit trap is exercised by its battery but not yet live-constructed inside the main loop. The *policy* layer above the forward pass (§2.5) and the capability-multiplier loop (§4), by contrast, *are* integrated and demonstrated live. The GPU mega-kernel that would fuse the forward-pass mechanisms into one persistent on-device kernel is a deliberate non-build (§7).

2.5 The scheduler-owned branching policy

A generation is full of *branch points*: how many diverse drafts to try, how many candidate tools to invoke, how many independent verifiers to run, how many lanes to interleave, whether to fetch a fact at all. The conventional answer bakes a constant into each call site (“try 3 samples”, “run 1 check”). We make the branching factor k a **scheduler-owned, dynamic decision**, on the same proposer/disposer spine as the rest of the kernel: the model (or a router) proposes route *scores*; the kernel *disposes* how many branches to execute.

The policy object, `KPolicy`, maps a small signal vector — uncertainty, an independent disagreement signal, host cost, task risk, importance — to a k with three non-negotiable properties:

- **Thin by default.** With low uncertainty, k collapses to the floor (one continuation, or zero — stay in-model). Spending is reserved for where it is earned. This operationalizes a hard-won empirical lesson: a heavy

reasoning protocol layered on a capable model *hurts* it; the kernel must be a capability *multiplier*, not a tax.

- **The calibration trap is honored.** The policy will not widen on a raw, uncalibrated confidence signal. Only a *calibrated* uncertainty or an *independent* disagreement signal (e.g., a branch that failed its hold-out gate — ground truth, not self-report) is allowed to drive widening; otherwise the policy stays thin and relies on validating the single branch. *Fail→pass is not proof*, applied to routing.
- **A risk safety-floor that cost cannot brake.** Discretionary widening (exploration) is braked by host cost; but a risk-mandated *verification* count is a floor that cost can never remove — a confidently-wrong model cannot talk the scheduler out of checking a risky branch. *k* is hard-clamped to a ceiling (anti-runaway: the model cannot enlarge its own branching budget), and a budget accountant charges every spend.

The cost input is the kernel’s own proprioception: a `sysinfo` sensor reports host load-per-core, memory pressure, scheduler queue depth, and GPU utilization, normalized by a *binding-constraint* rule (the scarcest resource governs). The kernel reads its own state to widen *k* when idle and stay thin when loaded.

An honest negative on the “free” uncertainty signal. A Mixture-of-Experts router computes, for free, a softmax over its experts; its entropy is an obvious candidate uncertainty signal, and we built an inert tap for it (bit-identical to base, `inv_moe_entropy`). When we *calibrated* it against deterministic ground truth, however, gate-softmax entropy did not predict failure — it predicted *success*, **inverted** (last-layer AUC 0.887, point-biserial $r = +0.66$, surviving a generation-length confound), and on a single task suite. Used naively as “high entropy → widen,” it would have been exactly backwards. The calibration gate did its job: we keep the entropy signal *unwired* and let the policy’s lever be validate-the-branch and independent disagreement. We *observe* the model’s internal expert-*k* as a signal; we *own* the external continuation-*k* as the decision.

The branch points the policy governs are enumerated with their live results in §6.7; they span escalation width, fact dispatch (§4.3), concurrency, verification depth, and capability proposal (§3.6).

3. The Self-Extending Deterministic Plane

The deterministic plane is not static. When the kernel encounters a task for which it has no proven primitive, it can *grow one* — and the discipline by which it does so without trusting the model is the heart of the contribution.

3.1 The hold-out gate

The hold-out gate is the kernel’s notion of proof for a pure primitive. Given a candidate function, a *generator* of random inputs, and an independent *oracle*, the gate runs the candidate on `n` freshly generated instances (default in the thousands) and accepts only if every output matches the oracle. Three design choices make it trustworthy rather than merely a large test:

- **The trust boundary is asymmetric.** The generator and oracle come from the *gap specification* — the trusted source of truth. The model supplies *only the candidate*. The verdict — running the oracle and comparing — is computed in the trusted parent process. The untrusted child never grades itself; it returns raw outputs and nothing else. This defeats the obvious attacks (a candidate that patches `__eq__`, that re-implements the oracle to agree with itself, or that forges a “passed” report on stdout).
- **A novelty floor.** The gate counts *distinct* instances and refuses to certify if the generator did not actually produce enough variety, closing the loophole where a degenerate generator makes “1000 instances” mean one instance tested 1000 times.
- **A placebo control.** Because a single fail→pass transition can be luck or leakage, the gate re-runs under an independent seed and pairs each acceptance with a placebo check; *fail→pass alone is never accepted as proof*.

The gate is hardened against hostile candidates: leaf comparisons are type-gated (a candidate cannot smuggle an always-equal object through structural comparison), and the gate catches `BaseException` (so a candidate raising `SystemExit` cannot crash the gate into a false pass).

3.2 Sandboxed execution

Candidate code must run somewhere, and that somewhere is the most security-sensitive surface in the system. The `venv` tool launches each candidate in a subprocess under a macOS `sandbox-exec` profile that denies network and denies file writes outside a scratch directory, in its own process group (so a fork bomb is killed by `killpg`), under CPU/memory limits, with bytecode writing disabled. The launched child computes *raw outputs only*; the parent generates the instances, runs the oracle, and compares. This division is what makes the sandbox sound: even a fully malicious candidate can at most return wrong numbers, which the gate then rejects — it cannot forge a verdict, because it never holds the verdict.

3.3 The extension pipeline

A `Gap` bundles everything needed to grow a primitive: an entry name, a natural-language description, sample cases, and the generator/oracle source. The `SelfExtender` drives the loop: synthesize a candidate, then `gate_candidate` — sample-test → hold-out gate → unsandboxed cross-check → second-seed re-gate → placebo — and, only on a clean sweep, register the primitive into the scheduler so it becomes callable *and* discoverable. Factoring `gate_candidate` into a single function ensures the pipeline and the escalation solver (§3.4) judge candidates *identically*.

3.4 The escalation solver

Real models fail on the first try. The escalation solver turns that failure into a ladder rather than a dead end. Each rung is judged by the same hold-out gate, so *no rung can win by memorizing the visible cases*:

Tier	Strategy	Rationale
1	Greedy, $T = 0$	One deterministic attempt.
2	k diverse attempts at $T = 0.8$	Temperature-0 retries are byte-identical; real diversity requires sampling.
3	Repair each diverse attempt, fed its own held-out failure	Targeted correction beats blind resampling.
4	Research: <code>web_search</code> + <code>fetch_page</code> , then build from the references	A knowledge gap may be closable with external material.
5	Abstain: “ <i>I’m sorry, I don’t know how to do that.</i> ”	An honest failure is preferable to a confident wrong answer.

The Tier-2 width is no longer a constant: it is disposed by the branching policy (§2.5) from an *independent* uncertainty — the severity of the Tier-1 hold-out-gate failure — gated by host cost and a budget. A clear, near-miss failure warrants few diverse attempts; a hard failure on an idle host warrants more; a saturated host stays thin. Every gate-proven solution is persisted and registered for reuse. The research tier feeds *untrusted* web text into the model’s prompt only — the resulting candidate must still clear the identical gate, so external content can influence *what* is proposed but never *whether* it is accepted.

3.5 Tools through the scheduler

All capabilities reach the model the same way: as scheduler-registered, capability-gated tools. The current catalog includes `datetime`; `sysinfo` (host memory/load/CPU/GPU and free disk *for the running partition only* by default, with broader exposure behind an explicit flag, and unconditional secret redaction; plus scheduler state — process table, queue depths, held processes, memory slots, grants); `trace` (read live or persisted scheduler telemetry and run an invariant self-check); `web_search` / `fetch_page`; a delegate-and-verify tool (fire an agent to run a tool, then reconcile its result against the trace, with the verdict computed kernel-side rather than self-reported); the memory tool of §5; and the fact-loop op of §4.3.

3.6 Capability proposal as a k-governed router

Before synthesizing a new primitive the kernel asks whether an *already-trusted* one fits. A lexical proposer ranks the routable registry (proven det primitives + syscall tools) by match to the need and emits scored candidates. The *disposal* of that ranking is k-governed rather than a fixed threshold: from the score margin (an independent signal) the policy decides how many top candidates to try — a clear winner routes at $k = 1$; a close race widens to the top-k, *each still gated by validate-the-branch* (the proposed primitive is actually run on the span’s cases, since a lexical score is uncalibrated); and when nothing clears the floor, the proposer returns *gap-detect* $\rightarrow$ *synthesize*, unifying “reuse” and “grow” under one branching decision.

4. The Capability-Multiplier Loop

The kernel’s most direct value to *accuracy* is not a new reasoning protocol but a verified fact delivered at the right moment. §4 describes that loop, the channel finding that justifies its design, and how it is wired live.

4.1 The loop

When the model faces a task it cannot do reliably but the deterministic plane *can* compute the answer to, the kernel runs a governed loop: take a base attempt; if it fails validation (an independent, ground-truth signal), ask the branching policy whether to spend; and if so, compute the verified fact with a hold-out-proven primitive, **deliver it to the model as tokens**, and re-prefill. The loop is **thin by default** — when the base attempt already validates, no fact is fetched and no second generation is run — and a load-bearing *inertness invariant* holds: with no facts injected, the loop is byte-identical to a plain generation. Cost and budget can still veto the spend (stay thin under load); a risk floor cannot be braked away.

4.2 Why tokens, not residuals: the channel finding

The natural temptation is to inject the verified fact directly into the residual stream (the Mode-B injector of §2.4), bypassing the token interface. We tested this carefully and report a clean negative bracketed by a clean positive.

Value delivery (works). A trained, result-conditioned residual write makes the model **emit** a verified value it cannot produce unaided: on a held-out evaluation the model’s output equals the injected value with deployment accuracy 1.0 and *causal* faithfulness 1.0 (the output tracks the injected value even when that value is deliberately wrong), a placebo injector collapses to chance, and the disabled injector is bit-identical to base. This generalizes: a single *value-only* injector (blind to the task) routes correctly on a held-out *task type* it never trained on (1.0), and multi-token values succeed via per-token re-injection (a two-digit result at full-value accuracy 1.0).

Fact use (fails through the residual channel). When the task instead requires the model to *reason with* the injected value — inject a secret digit, ask for a transform of it — residual injection fails: held-out-operation accuracy 0.0, even after training on related operations, and the same-injection/different-prompt discriminator at 0.0. A second, more principled attempt (inject early and at a single token position) also fails at 0.0, but is *diagnostic*: the model’s tendency to simply echo the injected value rises sharply, showing the value is *read but not used as an operand*. The decisive control: the **same fact delivered as tokens** yields 1.0 on the identical held-out operation. The bottleneck is the channel, not the model. Tokens are processed from layer 0, so the early layers turn the value into an operand; a mid-stack residual edit arrives too late and in the wrong representational basis.

The architectural conclusion is sharp and is what the loop is built on: **Mode B is a token-free value-delivery-as-output channel; a fact the model must compute *with* must enter as tokens (Mode A).**

4.3 The live dispatch

The loop is wired into a live path. A router answers “which verified fact does this prompt need?”, `dispatch` runs the governed loop or passes the prompt through unchanged (byte-identical when no fact applies), and `register_fact_loop_executor` exposes the whole thing as a capability-gated scheduler `op (op:fact_loop)` that admits, runs, and commits through the proof ledger like any other task — rejected fail-closed if its capability is not granted. The *propose* half can be lexical, but it is also available as the **learned in-weights syscall head**: a classifier on a mid-stack residual proposes the fact class, confidence-gated. On a routing

benchmark the learned head matches the lexical router on keyword-bearing prompts and *beats* it on keyword-free paraphrases the lexical matcher misses entirely — the learned-proposes / kernel-disposes loop, end to end: the head proposes the fact, the governed loop disposes the spend.

5. The Persistent Memory Store

The kernel is given a small, persistent memory with four properties the user specified: it is *always consulted*, *managed within hard bounds*, *kept up to date*, and *governed by a self-describing object that says so*. It is reached, like every other capability, through the scheduler as the capability-gated `op:memory`.

- **Persistent and durable.** SQLite in WAL mode, one file, surviving process restarts.
- **Hard-bounded (default 100 MB).** Every write is byte-accounted. If a write would exceed the cap, the store evicts least-recently-used *non-pinned* entries to make room; if it still does not fit — an oversized entry, or only pinned entries remain — the write is *rejected*. The cap is never exceeded, by construction.
- **A governance object.** A pinned, never-evicted entry holds the policy in plain language: *always consult this store before acting; record durable facts and proven helpers, update what changes, delete what becomes wrong; stay within the cap*. `consult()` always returns this object first — it is, literally, “the memory store object telling it to do so.” The governance object cannot be deleted.

The tool dispatches `consult / get / put / list / delete / stats`. Because eviction is LRU over non-pinned entries, the policy itself and any explicitly pinned facts are protected from pressure.

6. Validation

We report several independent bodies of evidence. Where a result depends on the live model it is labelled as such; the rest is deterministic and reproducible on CPU.

6.1 Forward-pass invariants (live model)

The eight invariant batteries of §2.4 all report residual $\equiv \mathbf{0}$ on the live North model: trap plane, KV park, switch-MLP isolation, speculate/rollback, deterministic interrupt, ring-cross rollback (including the >4096-token restore path), the Mode-B injector’s disabled identity, and the MoE-entropy tap’s inertness. Bit-exactness, not tolerance, is the bar — a single differing element fails the battery.

6.2 CPU test suites

Twenty-eight CPU suites pass in full (zero failures). The original kernel suites:

Suite	Checks	Suite	Checks
task_graph	26	tools	19
lane_oracles	43	orchestrator	17
trace_tool	15	holdout_gate	12
self_extend	14	venv_tool	14
memory_store	20	investigation	13
agent_delegate	10	escalate_solve	10
kv_park	10	entry_trap	9
web_tools	8	commit_kernel	7
scheduler	4	det_interrupt	3
commit_trap	all		

The nine suites added for the branching policy, hidden-state injection, and capability-multiplier work:

Suite	Checks	Concern
k_policy	34	the branching policy: calibration trap, risk floor, anti-runaway clamp
k_cost	15	sysinfo→cost (binding-constraint) + budget accounting
k_policy_ladder	11	k-governed escalation width
mode_b	8	hidden-state injector: disabled-identity, result-conditioning
fact_loop	43	the multiplier loop: inertness, thin-by-default, governance
fact_dispatch	43	the live dispatch: routing, pass-through, op:fact_loop, cap gate
fact_dispatch_head	26	learned-head routing
dynamic_k_seams	39	cost-governed concurrency + risk-governed verification depth
proposer_k	31	k-governed capability proposal + gap-detect

6.3 Coding benchmark through the kernel

To confirm that the kernel additions preserved end-to-end coding ability, we ran the full coding benchmark **through the kernel, on the live model via MLX**: 420 tasks across five suites. Result: **418/420**, with **zero trace self-check violations** reported by the analyzer on any suite.

Suite	Pass	Suite	Pass
warmups	49/50	ladder	100/100
challenge	199/200	expert	50/50
property	20/20		

The two misses were a single greedy draft on terse prompts; §6.4 shows the same model solves them generally once escalation and the gate are applied.

6.4 Solving held-out challenges without memorization (live model)

The two benchmark misses — `to_snake_case` (w47) and a Redis-RESP response parser (c156) — were re-attempted under the escalation solver, each judged by the hold-out gate at 2000 novel instances per seed across

two seeds, with a placebo control. Both were solved *generally*: **w47** at tier 1 (greedy), 2000/2000 on two seeds; **c156** at tier 2 (a temperature-0.8 diverse attempt), 2000/2000 on two seeds, after the original root cause was identified. The research tier was exercised end-to-end (a live `web_search` + `fetch_page` retrieved the genuine RESP specification before the gate ran). Because the generator and oracle are independent of the candidate and the instances are novel, passing the gate is evidence of a general algorithm, not of memorized cases.

6.5 Adversarial bug sweeps

New surface was audited by multi-agent sweeps in which independent reviewers proposed findings and a second wave attempted to *refute* each one (defaulting to “refuted” when unsubstantiated). The core-plane sweep (21 reviewers) verified fourteen findings; thirteen were fixed and regression-locked — among them the hold-out gate’s hostile-`__eq__` and `BaseException` paths, an admission-ceiling skippable via a null validator, the venv tool’s stdout-forge and unclamped resource limits, and the delegate tool’s self-graded verdict. An escalation sweep (4 reviewers) returned *ship* with zero CRITICAL/HIGH surviving. The capability-multiplier loop was reviewed the same way: a reviewer found the loop could mislabel an empty fact-provider as a “spend” (an audit that overstated what happened); it was fixed and locked, alongside fail-safe handling of a broken cost sensor or validator. The methodology is itself a kernel principle — *adversarial verification*: every claimed bug, and every claimed proof, is checked by an independent process biased toward refutation.

6.6 Grounding and calibration (live model, related result)

A related experiment gave a smaller model (Mellum) kernel-owned, read-only fixture investigation (`grep/read`, recon-seeded) on a 50-task cyber benchmark, lifting raw pass rate 22→25/50. The more important effect was *calibration*: nine of the remaining misses became honest “insufficient evidence” abstentions rather than confident wrong answers. This is the empirical seed of the capability-multiplier thesis: a verified fact lifts a model that can exploit it.

6.7 The branching policy and the capability multiplier (live model)

Two live North demonstrations close the loop from policy to pass-rate.

The branching policy adapts (live). With the policy wired into the escalation width, the disposed k on a hard task adapts to the kernel’s own state: idle $\rightarrow k = 5$, saturated host $\rightarrow k = 1$ (cost vetoes the spend), budget-capped $\rightarrow k = 2$; and when the base attempt already passes, the policy stays thin and the decision is never reached. The same policy governs four further seams, each tested: concurrency width (idle widens toward the lane count, a saturated host collapses to one, with multi-lane isolation independent of the chosen width); verification depth (a risk floor mandates extra independent checks that cost cannot brake away); fact dispatch (§4.3); and capability proposal (§3.6).

The capability multiplier (live). On a “count-then-use” slice the base model scores 0.05 unaided; with the verified count delivered as tokens it scores 0.475 — a $\sim 9.5\times$ lift — and the *governed* loop matches that ceiling while a saturated host vetoes the spend (back to base). We are explicit that the *thin-cost savings* are workload-dependent: when the base almost always fails, always-grounding is as cheap and as accurate; the governed loop’s advantage is realized on a mixed workload where many base attempts succeed. The multiplier itself (token-delivered verified facts) and the governance (cost/budget veto) are both demonstrated; the cost-savings are a function of the base-success rate.

Learned routing (live). The learned in-weights head routes the fact dispatch at 100% on a mixed benchmark versus 75% for the lexical matcher, recovering keyword-free paraphrases the lexical router misses entirely.

7. Discussion and Limitations

What the kernel buys. The recurring pattern is *learned-proposes / kernel-disposes* applied at four granularities: token/expert selection (the forward-pass mechanisms), the *branching factor* (the policy of §2.5), primitive synthesis (the hold-out gate), and task solving (the escalation ladder). In every case the model’s creativity is retained and its authority is removed; correctness, and now *how much to spend*, are adjudicated by a deterministic authority the model cannot influence. The capability-multiplier loop shows the payoff is concrete: a verified fact, delivered correctly, multiplies a weak model’s accuracy by an order of magnitude on the tasks where the deterministic plane can supply the missing operand.

Two honest negatives. First, the MoE gate-entropy signal is *inverted* under calibration and is therefore left unwired (§2.5) — a reminder that an “obvious” free uncertainty signal must be calibrated against ground truth before it is trusted, exactly the discipline the rest of the kernel enforces. Second, hidden-state *fact-use* fails through the residual channel (§4.2): a residual-injected value can be read but not reasoned with, while the same value as tokens succeeds at 1.0. These negatives are not incidental; they delineate the operating envelope of the two injection channels and justify the token-based design of the multiplier loop.

The mega-kernel: a nice-to-have, deliberately not built. The end-state in which decode, the deterministic function-blocks, and the scheduler all run as one persistent on-device (Metal) kernel is recorded in the design as a frontier item and *decided against* for now. Its payoff is *structural* (no CPU on the hot path, an on-GPU scheduler, in-kernel dispatch), not a decode-speed multiplier — the model’s decode is already GPU-bound — and its success invariant is the *already-proven* online==offline equivalence, merely fused. Against that thin, capability-neutral payoff stands a raw-Metal/ `MTLCommandBuffer` /C++ systems build that lives below MLX’s Python surface. We therefore reserve it for a genuine high-throughput *serving* regime or a systems-paper contribution, and note that the low-risk middle path — in-API step fusion (`mx.compile`, measured at 1.20×) plus function-block kernels (`mx.fast.metal_kernel`) — is available if decode latency ever becomes the bottleneck.

What is not yet integrated. The honest gap remains between the bit-exact forward-pass mechanisms (proven in isolation, §6.1) and a per-forward-pass commit trap in the running loop (which still schedules whole generations). The policy and dispatch layers above the forward pass *are* integrated and live; the deepest fusion is the deferred mega-kernel above.

Other limitations. The web tool’s SSRF surface is real and tracked. The persistent memory is single-process (SQLite WAL). The branching policy’s cost sensor reports memory pressure only when `psutil` is present (load/queue/GPU drive it otherwise). The capability multiplier’s thin-cost advantage is workload-dependent (§6.7). And the hidden-state fact-use negative leaves a research residue — multi-site or token-embedding-space injection — that we judge low-odds and have not pursued.

8. Conclusion

We have described a neural microkernel that treats a real MoE model as an untrusted proposer behind a trusted, deterministic disposer. The kernel admits work through a fail-closed three-gate contract; grows new *proven* capabilities at runtime via a hold-out gate that computes its verdict outside the model; escalates failures through a disciplined ladder; remembers within hard, self-governed bounds; and — new in this revision — makes the *branching factor itself* a scheduler-owned, calibrated, cost-aware decision, while a capability-multiplier loop turns a verified fact into an order-of-magnitude accuracy lift on tasks the model cannot do alone. The evidence — eight bit-exact forward-pass invariants, 28 fully-passing CPU suites, 418/420 on a kernel-routed coding benchmark, adversarial sweeps with all confirmed findings fixed, hold-out-proven solutions to previously-failed tasks, and live North demonstrations of the policy and the multiplier — supports the central claim at the level of mechanisms, the policy/dispatch layer, and scheduler-plane integration. We have also been careful to record what does *not* work (an inverted entropy signal, residual fact-use) and what we deliberately chose *not* to build (the GPU mega-kernel). The remaining work to make “interruptible, verified inference” a property of the running system, and not only of its parts, is to carry the proven forward-pass mechanisms into the live generation loop — a bridge whose two ends are now both built.

Appendix A. Reproduction

CPU validation is deterministic and self-contained:

```
cd finetune/gpt-oss-unsloth/scripts
PYTHONPATH="$PWD" python kernel/test_*.py          # 28 suites
```

Live-model results require the North weights and MLX. Selected drivers:

```
python kernel/run_coding_kernel.py --model north          # the
coding benchmark (§6.3)
python kernel/escalate_solve.py --task both --n 2000      # held-out
solving (§6.4)
python kernel/m5_dynamic_k_north.py                      # the
branching policy adapts (§6.7)
python kernel/run_fact_loop_north.py                     # the
capability multiplier ~9.5× (§6.7)
python kernel/run_fact_dispatch_head_north.py             # learned-
head routing (§4.3)
python kernel/inv_mode_b.py ; python kernel/inv_moe_entropy.py #
the two new ==0 batteries (§6.1)
```

Telemetry is written under `reports/telemetry/` and audited with `kernel/analyze_telemetry.py`.

Appendix B. Module map

Concern	Module
Scheduler ABI, admission gates	kernel/task_graph.py
Forward-pass commit trap	kernel/commit_trap.py
Hidden-state result injector (Mode B)	kernel/mode_b.py
MoE gate-entropy tap	kernel/moe_entropy.py
Branching policy + cost/budget	kernel/k_policy.py , kernel/k_cost.py
Verification-depth + concurrency k	kernel/verify_fanout.py , kernel/scheduler.py
Capability-multiplier loop	kernel/fact_loop.py
Fact dispatch + learned-head routing	kernel/fact_dispatch.py , kernel/fact_dispatch_head.py
Hold-out gate	kernel/holdout_gate.py
Sandboxed candidate execution	kernel/venv_tool.py
Self-extension pipeline + k-governed proposer	kernel/self_extend.py , kernel/lexical_proposer.py
Escalation solver	kernel/escalate_solve.py
Web-tool bridge	kernel/web_tools.py
Delegate-and-verify	kernel/agent_delegate.py
Syscall-tier tool registry	kernel/tools.py , kernel/tools_builtin.py
Trace / self-check	kernel/trace_tool.py , kernel/analyze_telemetry.py
Persistent memory	kernel/memory_store.py
Model adapter (MLX)	north_adapter.py